CUDA Assignment 1 Report

Saswata Mishra

August 2025

1 Introduction

The first assignment of the GPU programming course focused on writing folklore CUDA kernels to solve the problems in parallel, through the CUDA SM cores.

There were 5 problems given in total.

1.1) Basic versions of matrix multiplications with 1D launch configurations.

1.2) Basic versions of matrix multiplications with 2D launch configurations.

2) Tiled version of matrix multiplication.

3) Basic matrix transpose on GPU kernel.

4) Tiled version of matrix transpose.

I have attempted each problem, the corresponding codes can be found in the file: `main_<prob#>.cu`. To compile all the files in one go, you must run the `build.sh` script in the parent directory. It will create a `target` directory and a `target/results` sub directory.
You must run each binary code file from the parent directory like this:

```
./target/main_3 64 public_test_cases/matrix1.csv
```

This part is quite important for the evaluators. For more information regarding description of each source code file and their usage, I request them to refer to `readme.md`. It can be found inside the parent directory.

Note: I have generated a huge 1024 X 1024 size integer matrices and ran the test on them. There is a generator script written in Python which was used to generate the matrices.

2 Basic matrix multiplication

Here we do basic matrix multiplcation in CUDA. Here, each thread calculates an element in the resultant matrix. Below is a plot for the different block sizes observed:

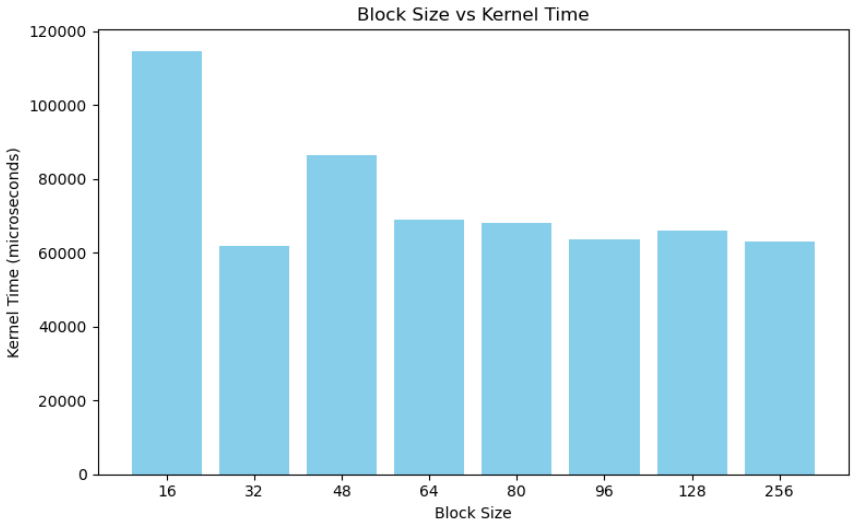

Figure 1: Performance for different block sizes in matrix multiplication.

Total number of threads here $= 1024 * 1024 = 1048576$

Trial No	Block Size, i.e., #Threads in a block (N)	#Thread Blocks (M)	Kernel Execution Time (μs)	Your insights on the execution time
1	16	65536	114619.938	
2	32	32768	61892.832	
3	48	21846	86374.789	
4	64	16384	68935.484	
5	80	13108	67915.391	
6	96	10933	63566.594	
7	128	8192	66029.211	
8	256	4096	63032.418	

Table 1: Kernel execution time for different block configurations.

Insight: Here we can see the performance getting better and better due to bigger block sizes as that results in better SM occupancy.

3 Kernel launch using 2D params

Here we launch the kernel with 2D launch configurations with dim3 data structure provided by CUDA framework. Like this:

```
1        // launch config definition:
2        dim3 grid(grid_size_x1, grid_size_y1);
3        dim3 block(block_size_x2, block_size_y2);
4        ...
5        gpu_mat_mul<<<grid, block>>> (d_matrix_1, d_matrix_2, d_matrix_result, no_of_rows_of_matrix_1,
             ↪ no_of_cols_of_matrix_1, no_of_cols_of_matrix_2);
```

Below is the plot for different block sizes in x and y dimenstion.

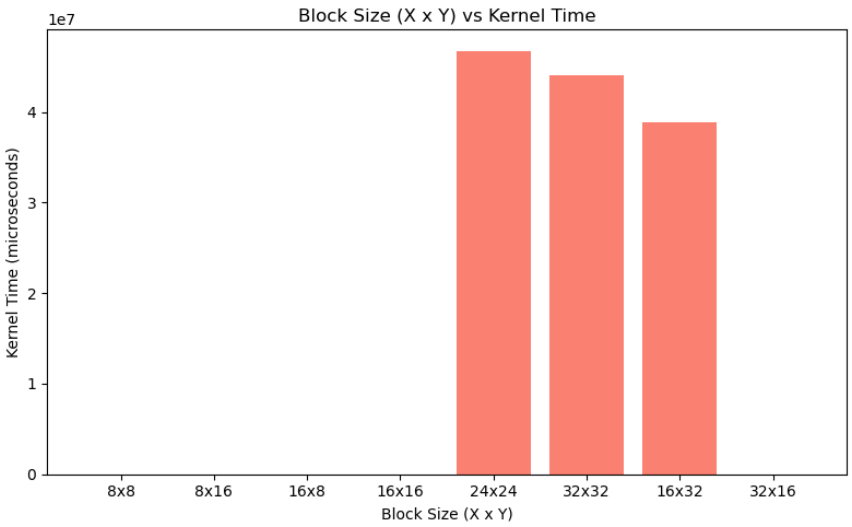

Figure 2: Performance for different block sizes in 2D launch config for matrix multiplication.

Trial No	Block Size, i.e., #2D block size (X2,Y2)	#2D grid size (X1,Y1)	Kernel Execution Time (μs)	Your insights on the execution time
1	8, 8	131072, 131072	24.608	
2	8, 16	131072, 65536	23.680	
3	16, 8	65536, 131072	21.984	
4	16, 16	65536, 65536	23.904	
5	24, 24	43690, 43690	46746768.000	
6	32, 32	32768, 32768	44033576.000	
7	16, 32	65536, 32768	38880464.000	
8	32, 16	32768, 65536	29.120	

Table 2: Kernel execution time for different 2D launch configurations.

Insight: As I am using NVIDIA Quadro P620, the GPU does not support 1024 threads per block efficiently. It has a small shared memory size, although it technically supports 1204 threads per block but that's notoriously slow due to repeated global memory access. That's the reason why we see those 3 huge spikes. But again, 32X16 config runs super fast due to memory coalescing.

4 Tiled Version of Matrix Multiplication

This is a strandard trick to reduce global memory usage by leveraging shared memory. NVIDIA GPU provides a shared memory which is shared inside a threadblock, i.e., all threads inside a block can access this memory, and the access time is way shorter than that of the global memory (GPU). The pseudocode is as follows:

```
    // Pseudocode for Tiled Matrix Multiplication

    define TILE_SIZE

    kernel matMulTiled(A, B, C, N):
        allocate shared As[TILE_SIZE][TILE_SIZE]
        allocate shared Bs[TILE_SIZE][TILE_SIZE]

        row = blockIdx.y * TILE_SIZE + threadIdx.y
        col = blockIdx.x * TILE_SIZE + threadIdx.x
        sum = 0

        for each tile t in range(0, N/TILE_SIZE):
            load A[row, t*TILE_SIZE + x] into As
            load B[t*TILE_SIZE + y, col] into Bs
            synchronize threads

            for k in 0..TILE_SIZE-1:
```

```
            sum += As[y][k] * Bs[k][x]
        synchronize threads

    C[row, col] = sum
```

Below are the observations for a $1024X1024$ sized matrix multiplication:

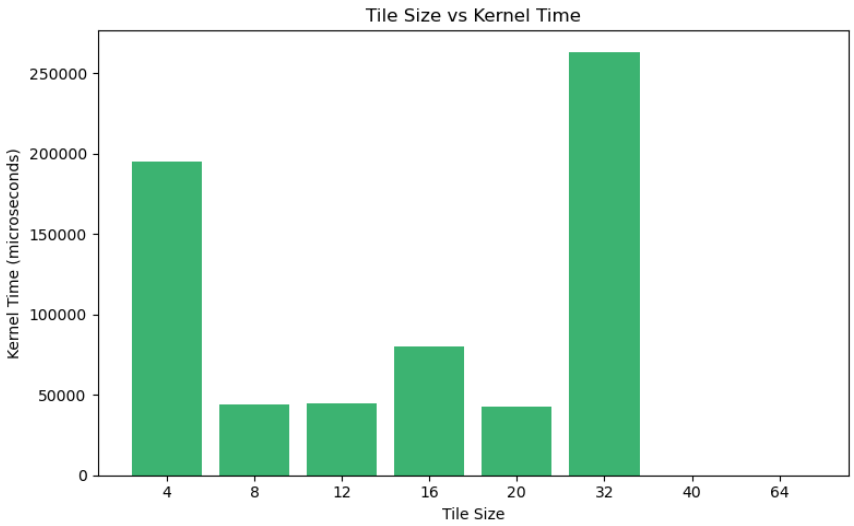

Figure 3: Performance for different tile sizes in tiled matrix multiplication.

Trial No	Tile Size	#Global Memory Access	#Shared Memory Access	Kernel Execution Time (μs)	Your insights on the execution time
1	4			195230.016	
2	8			43843.777	
3	12			44682.047	
4	16			79734.820	
5	20			42366.527	
6	32			263193.844	
7	40			38.112	
8	64			35.296	

Table 3: Kernel execution time for different tile sizes.

Insight:

5 Basic Version of Matrix Transpose

Here the idea is quite simple, each thread in the threadblock has a unique id which is defined by `id_1 = blockIdx.x*blockDim.x + threadIdx.x;`
in case of 1D launch config. And i and j are defined like this:

```
    unsigned int i = id_1 / cols;
    unsigned int j = id_1 % cols;
```

Now they can be used to transpose the matrix, each thread maps each cell of source matrix to the relevant cell of detination matrix.

```
    __global__ void gpu_tanspose(int *d_matrix_1, int *d_matrix_res, int rows, int cols) {

        unsigned int id_1 = blockIdx.x*blockDim.x + threadIdx.x;
        if (id_1 > (rows*cols - 1))
            return;

        // unsigned int id_2 = threadIdx.x*blockDim.x + blockIdx.x;
        unsigned int i = id_1 / cols;
```

```
            unsigned int j = id_1 % cols;

            d_matrix_res[j*rows + i] = d_matrix_1[i*cols + j];
        }
```

Below is the plot for different observations regarding different block sizes:

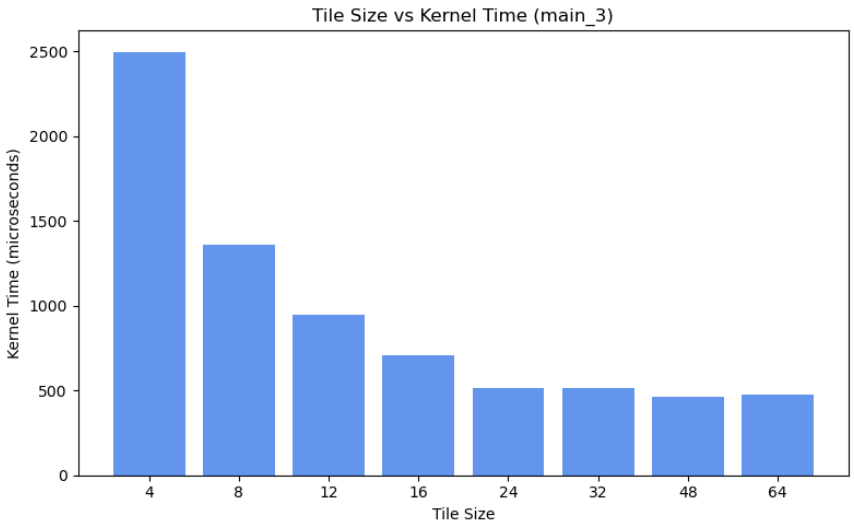

Figure 4: Performance for different block sizes in ordinary matrix transpose.

Trial No	Block Size, i.e., #Threads in a block (N)	#Thread Blocks (M)	Kernel Execution Time (μs)	Your insights on the execution time
1	4	256	2498.112	
2	8	128	1359.968	
3	12	86	946.080	
4	16	64	706.976	
5	24	43	512.384	
6	32	32	512.448	
7	48	22	462.976	
8	64	16	474.176	

Table 4: Kernel execution time for different block configurations.

6 Tiled Version of Matrix Transpose

Here we use the same ideology. We use shared memory to reduce global memory accesses, hence it leads to faster execution of the program.
Below is the plot of kernel execution time regarding different tile sizes:

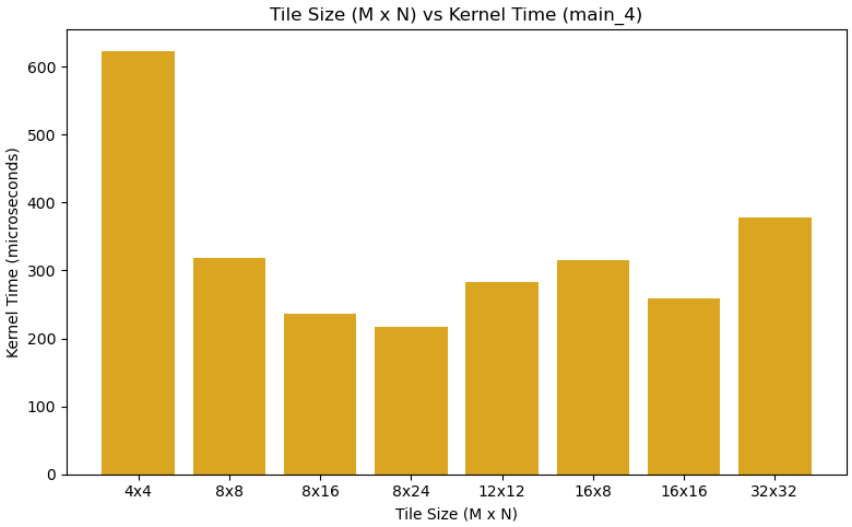

Figure 5: Performance for different tile sizes in tiled matrix transpose.

Trial No	**Tile Size M**	**Tile Size N**	**Kernel Execution Time (μs)**	**Your insights on the execution time**
1	4	4	623.264	
2	8	8	318.880	
3	8	16	235.520	
4	8	24	217.312	
5	12	12	283.520	
6	16	8	315.328	
7	16	16	258.496	
8	32	32	377.472	

Table 5: Kernel execution time for different tile configurations.

References

[1] NVIDIA, CUDA C Programming Guide. Available at: https://docs.nvidia.com/cuda/
[2] Matplotlib documentation: https://matplotlib.org/stable/index.html.
[3] GitHub documentation on How to write a README file